Neighbor-Interpolated Point Cloud Densification for 3D Gaussian Splatting Initialization

Thesis Research Draft

ABDUGANIEV ABDULVOSIT

Student ID: 24512169

Seoul National University of Science and Technology (SeoulTech)
Department of Integrated IT Engineering
Smart Media and Network Lab Master of Science
2026

Supervisor: Prof. Dong Ho Kim

GitHub: https://github.com/Abdulbaasit98/3dgs-pointcloud-densification

Draft – July 2, 2026

Contents

Abstract

3D Gaussian Splatting (3DGS) has emerged as a leading technique for real-time novel view synthesis, achieving state-of-the-art quality through a differentiable Gaussian rasterizer jointly optimized with a set of 3D Gaussians initialized from a sparse Structure-from-Motion (SfM) point cloud. A known limitation of this pipeline is sensitivity to the quality and coverage of the initial point cloud: regions under-sampled by SfM—such as textureless or reflective surfaces—begin training with insufficient Gaussian density and depend entirely on Adaptive Density Control (ADC) to compensate, which can slow early convergence.

This thesis proposes and evaluates a lightweight preprocessing step—*neighbor-interpolated point cloud densification*—that inserts new points at the midpoints of geometrically sparse neighbor pairs in the SfM point cloud before Gaussian initialization. The method requires zero modifications to the 3DGS training code, runs on CPU in seconds, and increases the initial point count by a controlled margin.

Experiments on the Tanks and Temples *truck* scene demonstrate consistent improvement in PSNR, SSIM, and LPIPS at every evaluated training checkpoint (ΔPSNR $= +0.39$ dB at iteration 1,000; $+0.17$ dB at 7,000), with the improvement most pronounced in early training and diminishing as ADC compensates. Per-view statistical analysis (Wilcoxon signed-rank, $p = 0.49$) indicates the effect, while consistent in aggregate metrics, does not reach significance at the per-view level with $n = 32$ views and a single scene. Final Gaussian counts were comparable between conditions (baseline: 1,681,300; densified: 1,733,031), indicating no compactness benefit. All code, data, and results are publicly available.

Keywords: 3D Gaussian Splatting, Novel View Synthesis, Point Cloud Initialization, Structure-from-Motion, Adaptive Density Control.

1 Introduction

Novel view synthesis—the problem of rendering a scene from arbitrary camera positions given only a set of training images—has undergone a dramatic transformation in recent years. Neural Radiance Fields (NeRF), introduced by Mildenhall et al. [2020], demonstrated that a continuous volumetric scene representation could be learned end-to-end from posed images using differentiable volume rendering, achieving unprecedented photorealism. Despite this success, NeRF's reliance on ray marching through a neural network makes real-time rendering computationally prohibitive.

3D Gaussian Splatting (3DGS), proposed by Kerbl et al. [2023], addresses this limitation by representing scenes as a collection of anisotropic 3D Gaussians rendered through a differentiable tile-based rasterizer. By coupling an explicit point-cloud representation with a fast GPU rasterizer, 3DGS achieves real-time rendering speeds (>100 FPS) while matching or exceeding NeRF-level quality on standard benchmarks.

A core aspect of the 3DGS pipeline that has received comparatively little attention is the *initialization* of the Gaussian set. The method initializes one Gaussian at the location of each point in a sparse 3D point cloud produced by Structure-from-Motion (SfM), typically COLMAP [Schönberger and Frahm, 2016]. Regions that are geometrically challenging for SfM—including textureless surfaces, specular materials, and weakly-observed scene boundaries—produce few or no reconstructed points, resulting in gaps in the initial Gaussian set.

This thesis investigates a simple, targeted approach: gap-filling point cloud densification via k-nearest-neighbor midpoint interpolation. Before the SfM point cloud is passed to the Gaussian initializer, a lightweight preprocessing pass identifies sparse neighbor pairs and inserts new points at their midpoints with interpolated color. This step is purely geometric, runs entirely on CPU in seconds, requires no neural components, and leaves the 3DGS training pipeline completely unmodified.

1.1 Contributions

- A gap-filling point cloud densification algorithm based on k-NN midpoint interpolation, implemented as a CPU-only preprocessing script compatible with the official 3DGS pipeline.
- A controlled experimental evaluation on the Tanks and Temples *truck* scene comparing standard vs. densified initialization across multiple training checkpoints, using PSNR, SSIM, and LPIPS.
- A per-view robustness analysis including Wilcoxon signed-rank and paired t-test statistical testing, and a compactness analysis of final Gaussian counts under both

conditions.

- A fully reproducible implementation with public code, notebook, and results at `https://github.com/Abdulbaasit98/3dgs-pointcloud-densification`.

2 Related Work

2.1 Neural Radiance Fields

Mildenhall et al. [2020] introduced NeRF, which encodes a scene as a continuous volumetric function $F_\theta : (\mathbf{x}, \mathbf{d}) \to (\mathbf{c}, \sigma)$ mapping a 3D position $\mathbf{x} \in \mathbb{R}^3$ and viewing direction $\mathbf{d} \in \mathbb{S}^2$ to color and density, optimized via differentiable volume rendering. Subsequent work improved training speed through hash-grid encodings [Müller et al., 2022] and extended NeRF to unbounded scenes [Barron et al., 2022] and dynamic scenes [Pumarola et al., 2021].

2.2 3D Gaussian Splatting

Kerbl et al. [2023] proposed 3DGS, representing scenes as a set of anisotropic 3D Gaussians. Each Gaussian $\mathcal{G}_i$ is parameterized by:

$$\mathcal{G}_i = \{\boldsymbol{\mu}_i,\, \boldsymbol{\Sigma}_i,\, \alpha_i,\, \mathbf{c}_i\} \tag{1}$$

where $\boldsymbol{\mu}_i \in \mathbb{R}^3$ is the center position, $\boldsymbol{\Sigma}_i \in \mathbb{R}^{3\times3}$ is the covariance matrix (decomposed as $\boldsymbol{\Sigma} = \mathbf{R}\mathbf{S}\mathbf{S}^\top\mathbf{R}^\top$ with rotation $\mathbf{R}$ and scaling $\mathbf{S}$), $\alpha_i \in [0, 1]$ is opacity, and $\mathbf{c}_i$ are spherical harmonic color coefficients.

The influence of a Gaussian at a 3D point $\mathbf{x}$ is:

$$G_i(\mathbf{x}) = \exp\!\left(-\tfrac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1}(\mathbf{x} - \boldsymbol{\mu}_i)\right) \tag{2}$$

Rendering proceeds via α-compositing of Gaussians sorted by depth:

$$\mathbf{C} = \sum_{i=1}^{N} \mathbf{c}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \tag{3}$$

Optimization uses a combined photometric and SSIM loss:

$$\mathcal{L} = (1 - \lambda)\mathcal{L}_1 + \lambda\, \mathcal{L}_{\text{SSIM}} \tag{4}$$

where $\lambda = 0.2$ in the original paper.

2.3 Initialization Sensitivity in 3DGS

Foroutan et al. [2024] systematically evaluated alternatives to SfM initialization for Gaussian Splatting, finding that initialization quality significantly affects convergence and final reconstruction quality. Jung et al. [2024] proposed RAIN-GS, which relaxes the accurate initialization constraint, allowing training to recover from low-quality point clouds through aggressive optimization. The present work takes the complementary direction: improving the initialization itself via fast geometric preprocessing, leaving the optimizer unchanged.

2.4 Point Cloud Densification

Classical point cloud upsampling methods include Moving Least Squares (MLS) surface fitting and deep learning approaches such as PU-Net. These target production of geometrically accurate dense surfaces. Our approach is simpler and more targeted: reducing large gaps in the SfM cloud to improve Gaussian coverage, using only pairwise distance geometry and linear color interpolation— no surface model is required.

2.5 Compact Gaussian Representations

LightGaussian [Fan et al., 2024] and Compact3D [Navaneet et al., 2023] propose pruning and quantization strategies to reduce the final Gaussian count. Scaffold-GS [Lu et al., 2024] introduces anchor-based Gaussians to reduce redundancy. These methods operate on the trained model, complementary to the present work. The compactness analysis in Section 5.4 directly examines whether initialization density affects the final Gaussian count.

3 Method

3.1 Problem Formulation

Let $\mathcal{P} = \{(\mathbf{p}_i, \mathbf{c}_i)\}_{i=1}^{N}$ denote the sparse SfM point cloud, where $\mathbf{p}_i \in \mathbb{R}^3$ is the 3D position and $\mathbf{c}_i \in [0, 255]^3$ is the RGB color of the i-th point. The 3DGS pipeline initializes one Gaussian at each $\mathbf{p}_i$. We seek to construct a denser point cloud $\mathcal{P}' = \mathcal{P} \cup \mathcal{P}_{\text{new}}$ such that the geometric coverage of the scene is improved, without access to additional images or depth sensors.

3.2 Gap Detection via k-Nearest Neighbors

For each point $\mathbf{p}_i \in \mathcal{P}$, we compute its k nearest neighbors using a k-d tree. Let $d_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|_2$ denote the Euclidean distance from point i to its j-th nearest neighbor.

We define the *scene-scale reference distance* as the median nearest-neighbor distance:

$$d_{\text{med}} = \text{median}_i \, d_{i,1} \tag{5}$$

A neighbor pair $(\mathbf{p}_i, \mathbf{p}_j)$ is classified as a *gap* if:

$$d_{ij} > \tau \cdot d_{\text{med}} \tag{6}$$

where $\tau > 1$ is the gap-threshold factor (hyperparameter). A higher τ results in fewer, more conservative insertions; a lower τ fills more aggressively.

3.3 Midpoint Interpolation

For each detected gap pair $(\mathbf{p}_i, \mathbf{p}_j)$, a new point is inserted at their geometric midpoint with linearly interpolated color:

$$\mathbf{p}_{\text{new}} = \frac{\mathbf{p}_i + \mathbf{p}_j}{2} \tag{7}$$

$$\mathbf{c}_{\text{new}} = \left\lfloor \frac{\mathbf{c}_i + \mathbf{c}_j}{2} \right\rfloor \tag{8}$$

To prevent excessive growth in sparse outlier regions, the number of new points inserted per original point is capped at $m_{\max}$ (hyperparameter). Each pair (i, j) is processed at most once (using a seen-pairs set).

3.4 Algorithm

Algorithm 1 Neighbor-Interpolated Point Cloud Densification

Require: Point cloud $\mathcal{P} = \{(\mathbf{p}_i, \mathbf{c}_i)\}$, neighbors k, threshold factor τ, max new per point $m_{\max}$

Ensure: Densified point cloud $\mathcal{P}'$

Build k-d tree T over $\{\mathbf{p}_i\}$
Compute d_{med} via Eq. (5)
$\mathcal{P}_{\text{new}} \leftarrow \emptyset$; seen $\leftarrow \emptyset$
for each point i **do**
    inserted $\leftarrow 0$
    for each neighbor j of i in T (in distance order) **do**
        if inserted $\geq m_{\max}$ **then break**
        end if
        if $d_{ij} \leq \tau \cdot d_{\text{med}}$ **then continue**
        end if
        if $(i, j) \in$ seen **then continue**
        end if
        seen $\leftarrow$ seen $\cup \{(\min(i, j), \max(i, j))\}$
        Compute $\mathbf{p}_{\text{new}}$, $\mathbf{c}_{\text{new}}$ via Eqs. (7)–(8)
        $\mathcal{P}_{\text{new}} \leftarrow \mathcal{P}_{\text{new}} \cup \{(\mathbf{p}_{\text{new}}, \mathbf{c}_{\text{new}})\}$
        inserted $\leftarrow$ inserted $+ 1$
    end for
end for
return $\mathcal{P}' = \mathcal{P} \cup \mathcal{P}_{\text{new}}$

3.5 Hyperparameters

Table 1 summarizes the hyperparameters used in this study.

Table 1: Hyperparameter settings used in the experiment.

Parameter	Value	Effect
k	6	Number of nearest neighbors considered per point
τ	3.0	Gap threshold multiplier; higher = fewer insertions
$m_{\max}$	1	Max new points per original point; limits growth

3.6 Integration with 3DGS

The method requires **zero changes** to the 3DGS training code. The `dataset_readers.py` loader in the official repository reads whichever `points3D.ply` file it finds in the scene's `sparse/0/` folder. Integration is therefore a file replacement:

1. Copy the original scene folder (preserving cameras and images unchanged).
2. Run Algorithm 1 on the original `points3D.bin`, writing the output as `points3D.ply`.
3. Train using the standard `train.py` command, pointing to the modified scene folder.

4 Experiments

4.1 Scene and Dataset

We evaluate on the *truck* scene from the Tanks and Temples benchmark [Knapitsch et al., 2017]: 251 training images, 32 held-out test views, approximately 136,029 points in the original SfM reconstruction.

4.2 Training Configuration

Table 2: Training configuration for both conditions.

Setting	Value
GPU	NVIDIA Tesla T4 (16 GB, Google Colab free tier)
Iterations	7,000 (paper uses 30,000; compute-constrained)
Test iterations	1,000 / 3,000 / 7,000
Loss	$(1-\lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{SSIM}}$, $\lambda = 0.2$
Random seed	Fixed (identical for both conditions)

4.3 Evaluation Metrics

We report three standard metrics on held-out test views:

- **PSNR** (Peak Signal-to-Noise Ratio, dB): higher is better.

$$\text{PSNR} = 10 \log_{10}\left(\frac{\text{MAX}^2}{\text{MSE}}\right) \tag{9}$$

- **SSIM** (Structural Similarity Index): higher is better, $\in [0, 1]$.
- **LPIPS** (Learned Perceptual Image Patch Similarity, VGG backbone): lower is better.

5 Results

5.1 Point Cloud Statistics

After applying Algorithm 1 with $k = 6$, $\tau = 3.0$, $m_{\max} = 1$:

$$d_{\text{med}} = 0.01129, \qquad \tau \cdot d_{\text{med}} = 0.03387 \tag{10}$$

Table 3: Point cloud statistics before and after densification.

Condition	Initial points	Increase
Baseline (original SfM)	136,029	—
Densified	197,869	+61,840 (+45.5%)

5.2 Convergence Speed

Table 4 reports PSNR, SSIM, and LPIPS on 32 held-out test views at matched training checkpoints. Δ values are densified minus baseline; positive ΔPSNR/ΔSSIM and negative ΔLPIPS indicate improvement.

Table 4: Quantitative results: baseline vs. densified initialization. Arrows indicate the direction of improvement. Δ = densified − baseline.

	PSNR (dB) ↑			**SSIM** ↑			**LPIPS** ↓		
Iter	Base	Dense	Δ	Base	Dense	Δ	Base	Dense	Δ
1,000	20.978	21.370	**+0.391**	0.7315	0.7538	**+0.0223**	0.3542	0.3264	**-0.0278**
3,000	22.728	22.966	**+0.238**	0.8074	0.8183	**+0.0108**	0.2524	0.2368	**-0.0156**
7,000	23.988	24.154	**+0.165**	0.8528	0.8595	**+0.0067**	0.1950	0.1846	**-0.0103**
Reference (baseline only, full 30k run):									
30,000	25.481	—	—	0.8850	—	—	0.1418	—	—

Densified initialization outperforms the baseline on all three metrics at every checkpoint. The advantage is largest at iteration 1,000 (ΔPSNR = +0.391 dB, ΔSSIM = +0.022, ΔLPIPS = −0.028) and narrows monotonically through iteration 7,000 (ΔPSNR = +0.165 dB). This convergence-speed pattern is consistent with the hypothesis that denser initialization provides an early-training head start that ADC gradually compensates for.

5.3 Per-View Robustness Analysis

To assess whether the aggregate improvement held consistently across test views, per-pixel ℓ_1 error was computed individually for each of the $n = 32$ test views at iteration

7,000. Let e_i^{base} and e_i^{dense} denote the mean ℓ_1 error for view i; define $\delta_i = e_i^{\text{dense}} - e_i^{\text{base}}$ (negative = densified better).

Table 5: Per-view ℓ_1 error analysis at iteration 7,000.

Statistic	**Value**
Views where densified is better ($\delta_i < 0$)	17/32 (53.1%)
Views where baseline is better ($\delta_i > 0$)	15/32 (46.9%)
Mean delta $\bar{\delta}$	-0.037
Standard deviation σ_δ	0.326
Best case (view 00014)	$\delta = -0.82$
Worst case (view 00003)	$\delta = +0.68$
Paired t-test: t-statistic	-0.634
Paired t-test: p-value	0.531
Wilcoxon signed-rank: p-value	0.488

Both the paired t-test ($p = 0.531$) and the Wilcoxon signed-rank test ($p = 0.488$) fail to reject the null hypothesis ($\alpha = 0.05$). The aggregate metric improvement, while consistent and reproducible, is not statistically significant at the per-view level given $n = 32$ views and a single scene/seed.

5.4 Compactness Analysis

Table 6 reports the final Gaussian count for both conditions at iteration 7,000.

Table 6: Final Gaussian counts at iteration 7,000.

Condition	**Init points**	**Final Gaussians**	**ADC added**	**Increase**
Baseline	136,029	1,681,300	+1,545,271	12.4×
Densified	197,869	1,733,031	+1,535,162	8.8×
Δ	+45.5%	+3.1%		

Despite a 45.5% larger initialization, the densified model contains only 3.1% more Gaussians at convergence. ADC added approximately 1.54×10^6 Gaussians in both conditions, suggesting its growth criteria respond to per-view residual error rather than global point density. The proposed method therefore confers **no compactness benefit**.

5.5 Summary

Note

Denser initialization produces a small, consistent improvement in aggregate reconstruction quality during early training (ΔPSNR $= +0.39$ dB at iter. 1,000). This effect is not statistically significant at the per-view level ($p \approx 0.49$, $n = 32$) and does not reduce the final Gaussian count. The contribution is best characterized as a modest, directional early-convergence benefit rather than a definitive quality improvement.

6 Limitations

- **Single scene:** Evaluation is limited to the Tanks and Temples *truck* scene. Generalization to other scene types is unknown.
- **Single seed:** All runs use a fixed random seed. Results may vary across seeds; the per-view analysis ($p \approx 0.49$) is inconclusive with $n = 32$.
- **Reduced iterations:** Training was capped at 7,000 iterations due to Google Colab free-tier session limits. The reference paper uses 30,000 iterations. The baseline's 30,000-iteration result is provided in Table 4 for reference only.
- **No hyperparameter sweep:** Only one setting of $(\tau, k, m_{\max})$ was evaluated due to compute constraints.
- **No surface geometry:** The midpoint interpolation in Eq. (7) does not account for surface orientation or local geometry, which may lead to off-surface insertions in curved regions.

7 Conclusion

This thesis proposed and evaluated neighbor-interpolated point cloud densification as a preprocessing strategy for improving 3D Gaussian Splatting initialization. The core contribution is a simple geometric algorithm (Algorithm 1) that identifies and fills gaps in sparse SfM point clouds using k-NN midpoint interpolation, requiring no changes to the 3DGS training pipeline.

Experiments on the Tanks and Temples *truck* scene demonstrated a consistent aggregate improvement across PSNR, SSIM, and LPIPS at every evaluated checkpoint (Table 4), with the largest advantage early in training (ΔPSNR $= +0.391$ dB at iteration 1,000) diminishing toward convergence. Two important null findings were also reported: (1) the improvement is not statistically significant at the per-view level (Wilcoxon $p = 0.488$, Table 5); and (2) the method provides no compactness benefit, as Adaptive Density

Control operates independently of initialization density (Table 6).

These findings position the method as a lightweight, zero-cost-at-training-time initialization improvement with a modest, directional benefit — a complete, honest, reproducible MSc-level experiment with clear directions for future work.

Future Work

- Multi-scene validation across the full Tanks and Temples and Deep Blending benchmarks to assess generalization.
- Hyperparameter sweep over $\tau \in \{2, 3, 4, 5\}$ and $k \in \{4, 6, 8\}$ to characterize the quality/compactness trade-off.
- Full 30,000-iteration runs to determine whether the early-training advantage persists at convergence.
- Surface-aware insertions using local normal estimation to improve geometric accuracy of synthetic points.
- Extension toward an IEEE Access submission with multi-scene, multi-seed statistical validation.

References

Barron, J. T., Mildenhall, B., Verbin, D., Srinivasan, P. P., and Hedman, P. Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. In *CVPR*, 2022.

Fan, Z. et al. LightGaussian: Unbounded 3D Gaussian compression with 15x reduction and 200+ FPS. In *NeurIPS*, 2024.

Foroutan, Y., Rebain, D., Yi, K. M., and Tagliasacchi, A. Evaluating alternatives to SfM point cloud initialization for Gaussian splatting. *arXiv:2404.12547*, 2024.

Jung, J., Han, J., An, H., Kang, J., Park, S., and Kim, S. Relaxing accurate initialization constraint for 3D Gaussian splatting. *arXiv:2403.09413*, 2024.

Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

Knapitsch, A., Park, J., Zhou, Q.-Y., and Koltun, V. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics*, 36(4), 2017.

Lu, T., Yu, M., Xu, L., Xiangli, Y., Wang, L., Lin, D., and Dai, B. Scaffold-GS: Structured 3D Gaussians for view-adaptive rendering. In *CVPR*, 2024.

Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., and Ng, R. NeRF: Representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020.

Müller, T., Evans, A., Schied, C., and Keller, A. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics*, 41(4), 2022.

Navaneet, K. L., Meibodi, K. P., Koohpayegani, S. A., and Pirsiavash, H. Compact3D: Compressing Gaussian splat radiance field models with vector quantization. *arXiv:2311.18159*, 2023.

Pumarola, A., Corona, E., Pons-Moll, G., and Moreno-Noguer, F. D-NeRF: Neural radiance fields for dynamic scenes. In *CVPR*, 2021.

Schönberger, J. L. and Frahm, J.-M. Structure-from-motion revisited. In *CVPR*, 2016.